

RAJMOHAN ARUMUGAM

Master Website Architecture

Multi-portal political leadership & public-service platform

Core concept: One master navigation experience leading into four independent digital portals, powered by one shared citizen-service and administration platform.

NEXT.JS · TYPESCRIPT · SUPABASE · CLOUDFLARE · AWS · DOCKER

Architecture rebuilt from the existing project architecture and expanded for the new requirements.

1. Product Structure

The platform is one application with five major experiences: a master navigation page plus four visually independent public portals. The portals share the technical platform, citizen-service engine, authentication, CMS and design-system foundation.

```
RAJMOHAN PUBLIC PORTAL
|
v
MASTER NAVIGATION / ENTRY PAGE
|
+-- SCHOOL EDUCATION
+-- TAMIL DEVELOPMENT
+-- INFORMATION & PUBLICITY
+-- MLA / EGMORE
|
+-- Shared Citizen Query Platform
+-- Shared Admin / CMS Platform
```

2. Master Navigation

Route: /

Primary asset: /public/images/navigation.jpg

The navigation page is the master entry experience. The existing navigation.jpg remains the visual foundation; the application overlays four interactive navigation portals around the central subject.

Four portals

01 — School Education

02 — Tamil Development

03 — Information & Publicity

04 — MLA · Egmore

The four cards are navigation nodes, not large content cards. Clicking one transitions into a complete department-specific website experience.

3. Shared Design System + Four Visual Systems

A master design system provides consistency, while each portal receives its own visual treatment.

Shared visual DNA: Instrument Serif, Instrument Sans, warm sandal/ivory, white, TVK-inspired maroon/red, TVK-inspired yellow, deep charcoal.

Shared components: navigation, buttons, typography primitives, query forms, authentication, accessibility, footer, cards and motion principles.

Portal-specific: hero composition, photography, section layouts, accent treatment, motion and storytelling.

4. School Education Portal

Routes: /en/school-education and /ta/school-education

Hero direction: Future · Learning · Students · Transformation.

Core sections: Hero, Education Vision, Current Priorities, Schools, Students, Teachers, Curriculum, Infrastructure, Initiatives, Education Timeline, News & Announcements, Resources, Citizen Query.

5. Tamil Development Portal

Routes: /en/tamil-development and /ta/tamil-development

Hero direction: Language · Literature · Heritage · Knowledge.

Core sections: Hero, Tamil Vision, Official Language, Literature, Awards, Nationalised Books, Sorkuvai, Students & Youth, Tamil Institutions, Research, Culture & Heritage, Global Tamil Engagement, News, Citizen Query.

6. Information & Publicity Portal

Routes: /en/information-publicity and /ta/information-publicity

Hero direction: Information · Communication · Media · Public Access.

Core sections: Hero, Latest Information, Press Releases, Minister Statements, Government Communication, Media, Publications, Video, Photo Archive, Announcements, Citizen Query.

7. MLA / Egmore Portal

Routes: /en/mla-egmore and /ta/mla-egmore

This portal acts as a digital constituency office rather than a departmental content site.

Core sections: Hero, About Egmore, My Representation, Constituency Development, Public Issues, Local Initiatives, Events, Citizen Services, News, Raise a Concern, Track Your Query, Contact.

8. Citizen Query Platform

The four portals share one centralized Citizen Service Engine. A citizen can submit a problem from any department and the system automatically associates the query with the selected department.

```
Department
↓
Raise a Query
↓
Category + Subject + Description
↓
Location + Attachments
↓
Mobile Number
↓
OTP Verification
↓
Query Created
↓
Reference ID
↓
Track Status
```

Query lifecycle

```
SUBMITTED
↓
RECEIVED
↓
UNDER REVIEW
↓
ASSIGNED
↓
IN PROGRESS
↓
RESOLVED
```

Optional statuses: NEEDS INFORMATION, REJECTED, CLOSED.

9. Mobile + OTP Authentication

Use Supabase Auth for phone authentication. The citizen enters a mobile number, receives an OTP, verifies it and receives an authenticated session. No traditional username/password account is required for citizens.

The authenticated citizen can access a dashboard containing their submitted queries, reference numbers, status timelines, attachments and updates.

10. Database Architecture

Recommended Supabase PostgreSQL entities:

- users
- citizen_profiles
- departments
- query_categories
- queries
- query_attachments
- query_status_history
- otp_sessions
- notifications
- profiles
- admins
- pages
- news
- events
- gallery
- media
- social_links
- site_settings

Core query fields

```
queries
- id
- reference_number
- citizen_id
- department_id
- category_id
- subject
- description
- location
- status
- priority
- assigned_to
- created_at
- updated_at
- resolved_at
```

11. Admin / Government Dashboard

Route: /admin

```
Dashboard
■■■ Overview
```

- Queries
 - ■■■ All
 - ■■■ School Education
 - ■■■ Tamil Development
 - ■■■ Information & Publicity
 - ■■■ MLA / Egmore
- News
- Events
- Pages
- Departments
- Gallery
- Documents
- Citizen Management
- Staff / Officers
- Analytics
- Settings

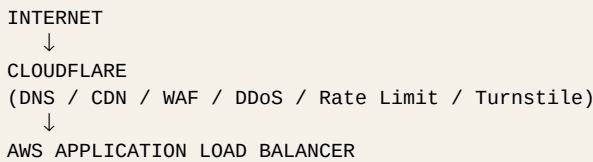
Role model

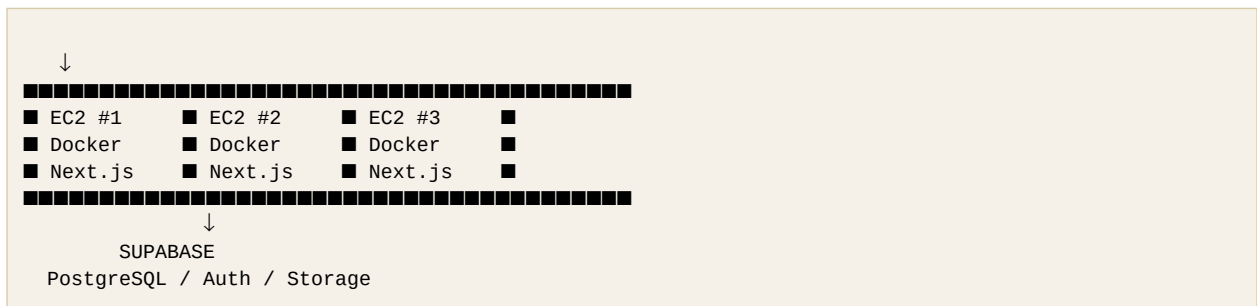
- SUPER_ADMIN — full access.
- CONTENT_ADMIN — pages, news and events.
- DEPARTMENT_ADMIN — queries for assigned department.
- MEDIA_ADMIN — gallery, images and videos.
- OFFICER — assigned queries and status updates.

12. Technical Stack

Layer	Technology	Purpose
Frontend	Next.js + TypeScript	Application and routing
UI	Tailwind CSS + shadcn/ui + Radix	Component system
Motion	GSAP + Motion + Lenis	Transitions and interaction
Forms	React Hook Form + Zod	Validated forms
Backend	Next.js Route Handlers / Server Components	Server-side logic
Database	Supabase PostgreSQL	Application data
Authentication	Supabase Auth	Citizen OTP + admin auth
Storage	Supabase Storage	Images, documents, media
Edge	Cloudflare	DNS, CDN, WAF, DDoS
Compute	AWS EC2	Next.js runtime
Load Balancing	AWS ALB	Traffic distribution
Containers	Docker	Identical deployments
Registry	Amazon ECR	Container images
CI/CD	GitHub Actions	Testing and deployment
Monitoring	Sentry + Cloudflare + CloudWatch	Observability

13. Production Infrastructure





Use an Auto Scaling Group so unhealthy instances can be replaced and additional capacity can be added when needed.

14. Static vs Dynamic

Static / heavily cacheable

Master navigation, department pages, biographies, vision, initiatives, news, events, gallery and other public content should be pre-rendered or cached aggressively where practical.

Dynamic

OTP, citizen authentication, query submission, query tracking, admin dashboard, admin authentication, status updates and personalized content require live backend requests.

15. Media Architecture

```

Supabase Storage
■■■ navigation
■■■ heroes
■■■ education
■■■ tamil-development
■■■ publicity
■■■ mla
■■■ news
■■■ events
■■■ gallery
■■■ documents

Future large video media → Cloudflare R2
  
```

Do not store large media files directly inside PostgreSQL.

16. Image Optimization

Process major images into responsive AVIF/WebP variants. Recommended widths: 400px, 800px, 1200px and 1920px. navigation.jpg is a critical first-screen asset and should be optimized without changing its intended composition.

17. Internationalization

Build Tamil and English from the beginning.

```

/en/
/ta/

/en/school-education
/ta/school-education

/en/tamil-development
/ta/tamil-development
  
```

Keep translations in structured content fields rather than mixing translations inside random components.

18. Security Architecture

```
Citizen
↓
Cloudflare WAF
↓
Turnstile
↓
Next.js API
↓
Zod validation
↓
Rate limiting
↓
Authorization
↓
Supabase RLS
↓
PostgreSQL
```

Never expose service-role keys, database passwords or Turnstile secrets to browser code. Sensitive database operations should be server-controlled and protected by Row Level Security.

19. Repository Structure

```
rajamohan-platform/
├── .github/workflows/
├── public/
│   ├── images/
│   │   ├── navigation.jpg
│   │   └── heroes/
│   │       ├── education/
│   │       ├── tamil/
│   │       ├── publicity/
│   │       ├── mla/
│   │       ├── news/
│   │       └── gallery/
│   ├── fonts/
│   └── icons/
├── src/
│   ├── app/
│   │   ├── [locale]/
│   │   ├── citizen/
│   │   ├── admin/
│   │   └── api/
│   ├── components/
│   ├── lib/
│   ├── data/
│   ├── config/
│   ├── styles/
│   ├── supabase/
│   │   ├── migrations/
│   │   └── seed.sql
│   ├── tests/
│   ├── scripts/
│   ├── Dockerfile
│   ├── docker-compose.yml
│   ├── next.config.ts
│   ├── tsconfig.json
│   └── package.json
```

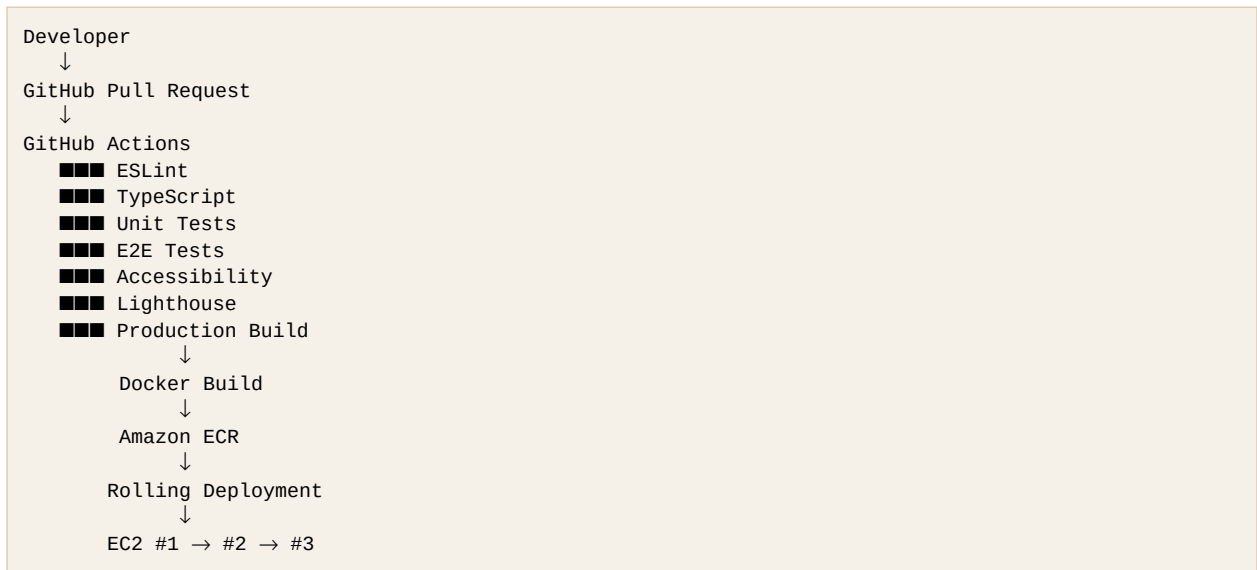
20. Design / Development Tooling

The project should actively use the two repositories already placed in the project folder:

- UI UX Pro Max Skill — design-system research, UI/UX rules, typography, color and stack-specific guidance.
- LibreUIUX Claude Code — UI/UX workflow and design-development assistance.

They should inform the implementation, while the project's own MASTER design system remains the final source of truth.

21. CI/CD



Use three environments: LOCAL → STAGING → PRODUCTION. Production deployment should occur only after CI checks and review.

22. SEO & Accessibility

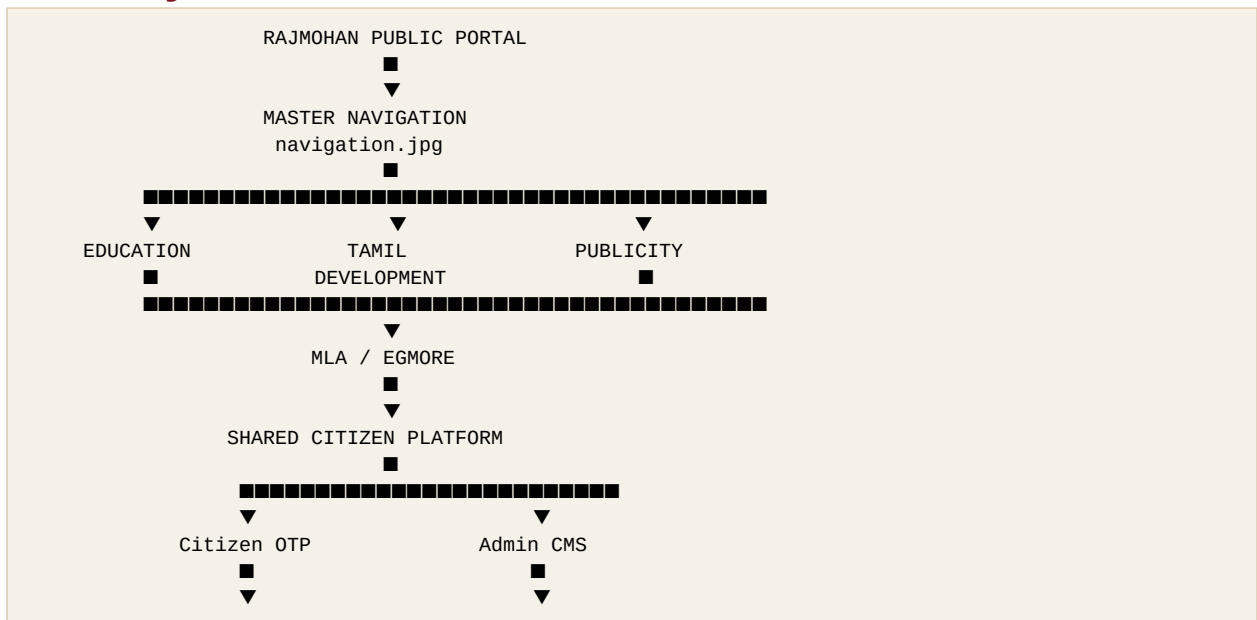
Each route should have independent title, description, canonical URL, Open Graph metadata, sitemap/robots coverage and appropriate structured data. Build keyboard navigation, semantic HTML, visible focus states, contrast compliance, reduced-motion support and accessible forms into the component system.

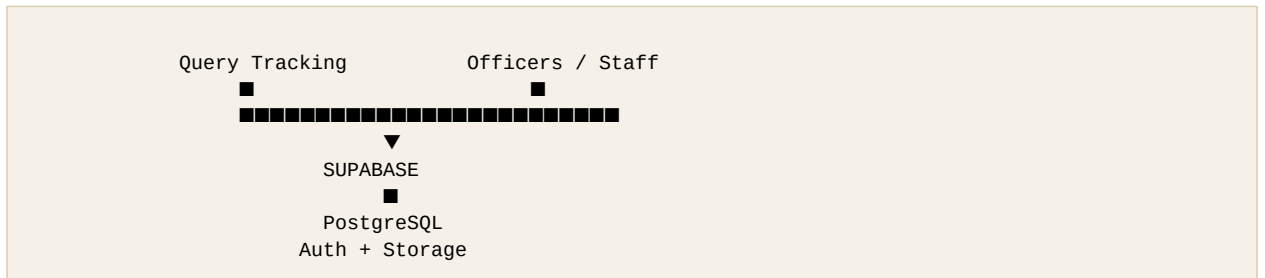
23. Monitoring & Backup

Monitoring: Sentry, Cloudflare analytics, AWS CloudWatch and uptime monitoring.

Backups: Supabase backups/PITR according to plan, GitHub source control and release tags, plus periodic media backup for important assets.

24. Final System Model





25. Architecture Decision

Do not build four separate Next.js projects. Build one production Next.js application with four isolated portal design systems and a shared backend platform. This provides the visual feeling of four independent websites while keeping authentication, citizen queries, CMS, database, security, deployment and maintenance centralized.

This is the recommended evolution of the existing architecture: retain the proven infrastructure foundation and add the citizen-service layer required by the new project scope.